

GitHub Repository klonen

create a repository on GitHub and clone it locally

2.1. Create a repository on GitHub

Sign in to GitHub. On the top right, click on the "+" symbol and create a new repository. Give it a valid name, e.g., . While you can leave all other settings as they are, it is good practice to:

supply a "description" one-liner and add a file.  README.md

If you create the repository with these added information, the first commit¹ will be created automatically for you: a file with the repository name as title and the description as text. If you click the  Edit icon to edit this file, you will notice it uses the Markdown syntax (more specifically, the Github Flavored Markdown variant).  README.md

In a later session, we will take a look at basic markdown syntax.

2.2. Clone a local copy of the repository with the terminal

Here, I present one way to work with git and GitHub - and possibly a different one in the live lecture. I do this on purpose, since git and GitHub workflows are a ultimately matter of individual taste.

For example, you can replicate every step of this notebook with the GUI of VS Code, i.e., you don't need to use a terminal application.

On your GitHub repository page, click on the green button that reads "Code" and copy the the "HTTPS" address of your repository, e.g. `https://github.com/jane-doe/webapp.git`

Next, open a terminal application, such as Git Bash on Windows (Git Bash comes bundled with your Git installation). Navigate with the `cd` command to the folder where you want the local repository copy to reside. For example, navigate to the folder. To clone  `C:\Users\me\projects\2` the repository, type in followed by the URL you just copied, such as: `git clone`

```
git clone https://github.com/jane-doe/webapp.git
```

If this is the first time using Git on your local machine, the Git Credential Manager should kick in and open a separate window asking for your GitHub credentials. Follow the workflow to sign in and allow the requested access.

You will see some status messages, which should finish off with some kind of success message, such as `.receiving objects: 100%, [...] done`

Congratulations! You have successfully created a local copy of your remote GitHub repository and established a connection between the two. Now, you can make changes to the repository offline and synchronize when you feel this makes sense (or once you're back online).

2.3. Maintain a file .gitignore

In an actively managed local repository, Git will propose to change-track any new files that are created in its folder (including sub-folders), thus adding them to the local repository and

subsequently pushing them to the remote repository via the stage-commit-push workflow (see next section).

For most projects it makes sense to exclude specific files and folders from version control. In a typical Python project, this includes Python-created caches, the Python Virtual Environment folder, local test databases or configuration information (e.g., secrets) that should not be shared.

To exclude specific files and folders, Git interprets the special file, typically placed in the root folder of your repository. To learn more about “.gitignore”, head over to this [GitHub Docs page](#).

 .gitignore

Here is an exemplary file for a typical Flask web application:  .gitignore

```
# Ignore __pycache__ folder (precompiled Python code)
__pycache__/
```

```
# Ignore venv folder (Python Virtual Environment)
venv/
```

```
# Optional: ignore instance folder (used by Flask web app, e.g., for SQLite database)
instance/
```

Do not miss this crucial step! The Python Virtual Environment folder (e.g.,) in particular will give you a very hard time if it's kept under source control. Some issues you will encounter if you don't exclude this folder:  venv/

The folder will become quite large over time, and changes to it will clutter your commit history.  venv/

A Python Virtual Environment set up on your machine will likely not work on another user's machine.

The file paths inside this folder might become too long for git to handle them properly, which will jeopardize the repository.

Everything in the folder can be recreated easily in local repositories, and does not need to be pushed to the remote repository:  venv/

Create a Python Virtual Environment with (on macOS) `python -m venv venv` `python3 -m venv venv`

Install the required Python packages with `pip install -r requirements.txt`

In your repository, you just need to maintain a that lists all Python packages that your Python application requires.  requirements.txt

You create the file with this command: `pip freeze > requirements.txt`

In the next section, we will switch to VS Code. However, everything can also be performed from the terminal application (see this handy [Git cheat sheet](#) listing the most common commands to follow up). Of course, the other way is also possible: You could have cloned the repository from within VS Code in the first place, without ever touching a terminal application like Git Bash.

3. Apply changes to your repository

3.1. Create your first commit with VS Code

Start VS Code, then press ++ / ++, then navigate to the folder containing the local repository. `Ctrl` `K` `⌘` `K`

Open , add some text to it and save the file with + / +. Press ++ / ++ to open the "Source Control" side pane. In the "Message" field, type in a description of what the change will do once applied. 
README.mdCtrlS⌘ShiftCtrlGShift⌘G

Writing "good" commit messages is an art in itself. If you want to get down that rabbit hole, head over to this page (if you make using Git a habit, reading that guideline is time well-spent).

Many practitioners adhere to three basic guidelines:

Keep the commit message short, not longer than 60 characters.

Write in present tense what the commit does to the repository.

It follows from 1.+2. that an individual commit contains focused, incremental work: it won't turn the entire repository upside-down.

Press + / + to commit the change. You will be asked whether all changes shall be stagedCtrlEnter⌘Enter3 and committed in one go. Agree to this suggestion. Now you have updated the local repository with the change to the file.  README.md

Next, click the "Sync Changes" button and confirm: this will pull4 any changes from the remote repository to your local repository (there are none at this moment), and in a second step push5 your local commit to the remote repository (the change you just committed).

Return to the remote repository on GitHub (refresh the browser tab) to confirm that your commit has been pushed from local repository to remote repository.

3.2. Apply a quick fix to a committed change

At times, after a commit you notice a small mistake that you want to fix quickly - maybe a typo, or you want to include another file to the commit. Fortunately, Git lets you easily change the last commit:

Just fix whatever needs fixing. On the VS Code "Source Control" side pane click on the small "..." icon ("More Actions..."), then "Commit" and "Commit All (Amend)". An editor tab opens that lets you change the commit message. Close the tab to commit the fix.

To perform the same action with Git terminal commands, first stage the change:

```
git add .
```

Next, commit with the flag:--amend

```
git commit --amend
```

An editor tab (or editor window) opens that lets you change the commit message. Close the editor tab (or window) to to commit the fix.

3.3. Restore to an earlier repository state

Since Git keeps track of all the changes applied to the repository, it can automatically create a commit for you that restores the repository to an earlier state. (Think of this as a big undo button.)

Suppose we want to undo a change we applied before, say to the file. First, open that file. 
README.md

From the VS Code “Explorer” side pane (⇧ / ⇧), open the “Timeline” section, then “Filter Timeline” for “✓ Git History”.⇧Ctrl⇧Shift⌘E

Click on an earlier commit to open a side-by-side comparison between current and older state (i.e., the diff view). If you found the correct commit, right-click on the timeline item and select “Copy Commit ID”.

Open a terminal with ⇧ / ⇧, . Revert to the previous state with ⇧, for example:⇧Ctrl⇧Shift⌘Pterminal create new terminalgit revert {commit ID}

```
git revert c6e0eed92329d14417faf7c26b3ca8fd7a1c9ccb
```

Obviously, the commit ID will be different in your case.

An editor tab will open that lets you change the proposed commit message. In most cases you want to simply close the editor. Finally, a new commit is created with all changes applied such that in effect you have gone back in history.

The feature is very strong motivation to keep individual commits focused, often limited to just one file.git revert

If you want to restore just a particular file out of a commit that affects multiple files, call ⇧. With the flag, Git computes the changes and stages them, but does not commit them yet. This gives you the chance to un-stage and discard any changes that you don’t want applied, before confirming the commit.git revert -n {commit ID}-n

4. Apply a basic GitHub workflow in VS Code

4.1. A basic GitHub workflow

GitHub adds a layer of collaboration functions on top of Git. Consider the following diagram:

```
main
feature
joe commit m-1
joe commit m-2
jane commit f-1
joe commit f-2
pull-request-1
rick review f-1
comment
jane review f-2
rick review f-3
ready to merge
v1
```

In its most basic form, the GitHub collaboration workflow discourages direct commits to the remote repository. Instead, the workflow suggests to

```
create a branch6 (e.g., called feature)
make one or multiple commits to that branch,
file a pull request7 to propose applying the branch changes to the branch,main
discuss the pull request, possibly making additional commits in the process,
```

(dis-)approve to merge the branch commits with the branch, and finally main apply the change and delete the branch. feature
It is good practice to create and maintain a file that specifies the particular collaboration workflow for your repository. The Mozilla Science Lab has published a great instruction to get you started on this important topic.  CONTRIBUTING.md

icch hab jz ein repository erstellt, was jz?
Mehr anzeigen Weniger anzeigen

Perfekt, du hast den wichtigsten Schritt schon gemacht 👍 Jetzt geht's weiter mit dem **lokalen Klonen und Arbeiten**. Ich führe dich Schritt für Schritt durch – ganz praktisch:

✓ 1. Repository auf deinen PC klonen

1. Geh auf dein GitHub-Repository
2. Klick auf den grünen „Code“ Button
3. Kopiere die **HTTPS-URL** (sieht so aus):

```
https://github.com/dein-name/webapp.git
```

4. Öffne dein Terminal (z. B. **Git Bash** oder VS Code Terminal)
5. Geh in den Ordner, wo du das Projekt speichern willst:

```
Bash
cd C:\Users\deinName\Documents
```

6. Klonen:

```
Bash
git clone https://github.com/dein-name/webapp.git
```

👉 Danach hast du einen neuen Ordner **webapp** auf deinem PC.

✓ 2. Projekt in VS Code öffnen

```
Bash
cd webapp
code .
```

(Falls `code .` nicht geht → einfach VS Code öffnen und den Ordner manuell auswählen)

✓ 3. .gitignore erstellen (sehr wichtig!)

Erstelle im Hauptordner eine Datei:

```
.gitignore
```

Inhalt für dein Flask/Python-Projekt:

```
__pycache__/  
venv/  
instance/
```

👉 Das verhindert später viele Probleme!

✓ 4. Virtual Environment erstellen

Im Terminal:

```
Bash  
python -m venv venv
```

Aktivieren:

- Windows:

```
Bash  
venv\Scripts\activate
```

✓ 5. Erste Änderung machen + committen

1. Öffne die **README.md**
2. Schreib z. B.:

```
Mein erstes Webapp Projekt 🚀
```

3. Speichern (Strg + S)

4. In VS Code:

- Links auf **Source Control (Git Symbol)**
- Nachricht schreiben:

```
update README
```

- Auf **Commit** klicken

5. Danach:

👉 **Sync Changes** klicken (oder Push)

✓ 6. Fertig 🎉

Jetzt hast du:

- GitHub Repo ✓
- Lokale Kopie ✓
- Erste Änderung gepusht ✓

👉 Was kommt als Nächstes?

Typischer nächster Schritt:

- Flask App reinpacken (dein Code von vorher)
- `requirements.txt` erstellen:

Bash

```
pip freeze > requirements.txt
```

Wenn du willst, kann ich dir jetzt **genau zeigen**, wie du deine Flask-App richtig in das Repo reinpackst und startklar machst 🚀

```
(venv) PS C:\Users\bunco\Documents\webapp> flask run
* Debug mode: off
WARNING: This is a development server. Do not use it in
a production deployment. Use a production WSGI server
instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [10/Apr/2026 12:14:55] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [10/Apr/2026 12:14:55] "GET /favicon.ico
HTTP/1.1" 404 -
127.0.0.1 - - [10/Apr/2026 12:15:04] "GET / HTTP/1.1" 200 -
(venv) PS C:\Users\bunco\Documents\webapp>
```

was soll ich jz mit dem link machen

Der Link ist dein **lokaler Webserver** 👉

👉 <http://127.0.0.1:5000>

Das bedeutet:

- ➔ Deine Flask-App läuft gerade auf deinem Computer
- ➔ Du musst den Link einfach im **Browser öffnen**

✅ Was du jetzt machen sollst:

1. Kopiere den Link:

```
http://127.0.0.1:5000
```

2. Öffne deinen Browser (Chrome, Edge, etc.)

3. Füge den Link oben in die Adresszeile ein

🔍 Was dann passiert:

- Deine Web-App wird angezeigt 🎉
- Alles, was du in `app.py` programmiert hast, siehst du dort

! Kleine Hinweise zu deinem Screenshot:

- **WARNING: This is a development server** → **vollkommen normal**
- **GET / 200** → **alles funktioniert**
- **favicon.ico 404** → **egal**, kein Fehler (nur Icon fehlt)

👉 Wichtig:

Der Link funktioniert **nur solange dein Terminal läuft**

👉 Wenn du das Terminal schließt → Seite geht nicht mehr

Wenn du willst, zeig mir kurz deinen `app.py` Code, dann sag ich dir:

👉 ob alles korrekt ist

👉 und wie du als Nächstes deine App erweitern kannst 🚀

das war der link <https://github.com/22burcu/webapp.git>

Ahhh okay 😊 jetzt verstehe ich – das ist dein **GitHub-Link**, nicht der Flask-Link.

👉 Dein Repo:

<https://github.com/22burcu/webapp.git>

🔥 Was du damit jetzt machst:

✓ 1. Lokal klonen (falls noch nicht gemacht)

In deinem Terminal:

```
Bash
git clone https://github.com/22burcu/webapp.git
```

👉 Danach:

```
Bash
cd webapp
```

✓ 2. Prüfen ob Git verbunden ist

Im Terminal:

```
Bash
git status
```

Wenn sowas kommt wie:

```
On branch main
nothing to commit
```

👉 Perfekt 👍

✓ 3. Deine aktuellen Dateien pushen (sehr wichtig!)

Du hast ja schon:

- `app.py`
- `venv`
- `__pycache__`

Jetzt:

```
Bash
git add .
git commit -m "add flask app"
git push
```

! GANZ WICHTIG (sonst bekommst du Probleme):

Du hast `venv` und `__pycache__` drin → das darf NICHT auf GitHub!

👉 Mach das jetzt sofort:

1. `.gitignore` erstellen

Datei:

```
.gitignore
```

Inhalt:

```
venv/  
__pycache__/
```

2. Dann nochmal:

```
Bash  
git add .  
git commit -m "add gitignore"  
git push
```

Kurz gesagt:

- GitHub-Link → für Code speichern & teilen
- Flask-Link (127.0.0.1) → für deine Website

Nächster Schritt

Willst du:

1. Deine Flask-App online stellen (nicht nur lokal)? 
2. Oder erstmal sauber strukturieren (templates, HTML etc.)?

Sag einfach 

On branch main nothing to commit

```
(venv) PS C:\Users\burcu\Documents\webapp> git clone https://github.com/22burcu/web  
app.git  
Cloning into 'webapp'...  
remote: Enumerating objects: 3, done.  
remote: Counting objects: 100% (3/3), done.  
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)  
Receiving objects: 100% (3/3), done.  
(venv) PS C:\Users\burcu\Documents\webapp> cd webapp  
(venv) PS C:\Users\burcu\Documents\webapp\webapp> |
```

so sieht es aus

Perfekt  das sieht **genau richtig** aus!

 Was passiert ist:

- Du warst schon im Ordner **webapp**

- Dann hast du nochmal `git clone` gemacht
- Deshalb hast du jetzt **einen Ordner im Ordner**:

```
webapp/webapp
```

🔥 Wichtig: Du bist gerade hier:

```
C:\Users\burcu\Documents\webapp\webapp
```

👉 Das ist dein **echtes Git-Projekt** jetzt

✅ Check ob alles passt

Gib ein:

```
Bash  
git status
```

Wenn da steht:

```
On branch main  
nothing to commit
```

👉 Dann ist alles korrekt verbunden 🎉

⚠️ Kleiner Fehler (nicht schlimm)

Du hast jetzt:

```
webapp  
└── webapp
```

👉 Das ist unnötig doppelt

💡 Saubere Lösung (empfohlen)

Geh einen Ordner zurück:

```
Bash  
cd ..
```

Dann lösche den äußeren Ordner **ODER** arbeite einfach im inneren weiter.

- 👉 Einfachste Lösung:
- ➡️ Bleib jetzt einfach in:

```
webapp\webapp
```

🚀 Was du JETZT machen sollst:

1. Schau ob deine Dateien da sind

```
Bash  
dir
```

Du solltest sehen:

- README.md (von GitHub)
- (evtl. noch nichts anderes)

2. Jetzt deine Flask App reinpacken

👉 Zieh deine `app.py` in diesen Ordner
(oder erstelle sie neu)

3. Dann:

```
Bash  
git add .  
git commit -m "add flask app"  
git push
```

! GANZ WICHTIG (wirklich wichtig 😄)

Mach sofort `.gitignore`:

```
venv/  
__pycache__/
```

Dann:

```
Bash  
git add .  
git commit -m "add gitignore"  
git push
```

🧠 Kurz gesagt:

- ✓ Git funktioniert
- ✓ Repo ist verbunden
- ✓ Du bist ready

Wenn du willst, kann ich dir jetzt zeigen:

- 👉 wie du eine **richtige Flask Website mit HTML (templates)** baust oder
- 👉 wie du sie **online stellst (nicht nur lokal)** 🌐

```
(venv) PS C:\Users\burcu\Documents\webapp\webapp> .gitignore venv/
>> __pycache__/
.gitignore : Die Benennung ".gitignore" wurde nicht als Name eines Cmdlet, einer
Funktion, einer Skriptdatei oder eines ausführbaren Programms erkannt. Überprüfen
Sie die Schreibweise des Namens, oder ob der Pfad korrekt ist (sofern enthalten),
und wiederholen Sie den Vorgang.
In Zeile:1 Zeichen:1
+ .gitignore venv/
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (.gitignore:String) [], CommandNotF
oundException
+ FullyQualifiedErrorId : CommandNotFoundException

__pycache__/ : Die Benennung "__pycache__/" wurde nicht als Name eines Cmdlet,
einer Funktion, einer Skriptdatei oder eines ausführbaren Programms erkannt.
Überprüfen Sie die Schreibweise des Namens, oder ob der Pfad korrekt ist (sofern
enthalten), und wiederholen Sie den Vorgang.
In Zeile:2 Zeichen:1
+ __pycache__/
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (__pycache__:String) [], CommandNo
tFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

(venv) PS C:\Users\burcu\Documents\webapp\webapp>
Mehr anzeigen Weniger anzeigen
```

Ahh 😊 ich sehe genau, was passiert ist — du hast versucht **.gitignore wie einen Befehl im Terminal** auszuführen. Das geht nicht.

- 👉 **.gitignore ist eine Datei**, kein Command.

✓ So machst du es richtig (ganz einfach)

♦ Methode 1 (empfohlen – über VS Code)

1. Links im Explorer (wo du `app.py` siehst)
2. Rechtsklick → **New File**
3. Name eingeben:

```
.gitignore
```

4. Inhalt reinschreiben:

```
venv/  
__pycache__/
```

5. Speichern (Strg + S)

♦ Methode 2 (Terminal – wenn du willst)

```
Bash  
New-Item .gitignore
```

Dann öffnen:

```
Bash  
notepad .gitignore
```

Und reinschreiben:

```
venv/  
__pycache__/
```

✅ Danach GANZ wichtig:

```
Bash  
git add .  
git commit -m "add gitignore"  
git push
```

⚠ Extra wichtig (weil du venv schon hast!)

Falls `venv` schon getrackt wird:

```
Bash  
git rm -r --cached venv  
git rm -r --cached __pycache__
```

Dann nochmal:

```
Bash  
git add .  
git commit -m "remove venv and cache"
```

🧠 Warum das wichtig ist:

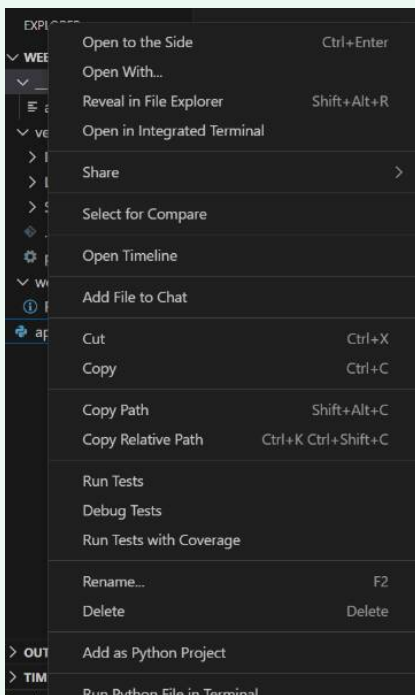
Ohne `.gitignore`:

- ❌ lädst du unnötige Dateien hoch
- ❌ dein Repo wird riesig
- ❌ kann später kaputt gehen

Wenn du willst, schick mir kurz dein VS Code Screenshot danach → ich check ob alles sauber ist ✅

So machst du es richtig (ganz einfach) ----- ### ♦ Methode 1 (empfohlen – über VS Code)

1. Links im Explorer (wo du `app.py` siehst) 2. Rechtsklick → **New File** 3. Name eingeben: `.gitignore`



gibts nicht

Ahh ja 😊 du hast **auf eine Datei rechtsgeklickt**, deshalb siehst du „New File“ nicht.

👉 Du musst auf den **Ordner oben** klicken, nicht auf `app.py`.

✅ So geht's richtig:

♦ Schritt für Schritt:

1. Links im Explorer:

👉 Rechtsklick auf **den Ordner „webapp“** (ganz oben)

2. Dann siehst du:

👉 **New File** ✓

3. Klick drauf und gib ein:

```
.gitignore
```

📸 Warum du es nicht siehst:

Du hast hier geklickt:

👉 auf **app.py** → da gibt's kein „New File“

Du musst hier klicken:

👉 auf den Ordner **WEBAPP**

🔥 Alternative (ultra easy):

Wenn es immer noch nicht klappt:

👉 Oben im Explorer:

- Klick auf das kleine 📄 **Icon (New File)**

Dann:

```
.gitignore
```

✍️ Inhalt rein:

```
venv/  
__pycache__/
```

Speichern ✓

🚀 Danach im Terminal:

```
Bash  
git add .  
git commit -m "add gitignore"  
git push
```

Wenn du willst, schick mir nochmal Screenshot danach – ich sag dir direkt ob alles perfekt ist 🍌

kann man repository umbenennen

Ja, kannst du ganz easy machen 👍

✅ Repository auf GitHub umbenennen

1. Geh auf dein Repository:
👉 <https://github.com/22burcu/webapp>
2. Klick oben auf **Settings** ⚙️
3. Ganz oben siehst du den Namen (z. B. **webapp**)
4. Ändere ihn → z. B.:

```
mein-flask-projekt
```

5. Klick auf **Rename**

⚠️ Wichtig danach!

GitHub ändert den Link automatisch, z. B.:

```
ALT: https://github.com/22burcu/webapp.git  
NEU: https://github.com/22burcu/mein-flask-projekt.git
```

🔧 Lokal musst du den Link updaten

Im Terminal:

```
Bash  
git remote -v
```

Dann:

```
Bash  
git remote set-url origin https://github.com/22burcu/NEUER-NAME.git
```

Beispiel:

```
Bash  
git remote set-url origin https://github.com/22burcu/mein-flask-projekt.git
```

💡 Gute Nachricht

👉 GitHub leitet alte Links meistens automatisch weiter
ABER: sauberer ist es, es lokal anzupassen

🧠 Tipp

- Keine Leerzeichen im Namen ❌
- Lieber:
 - `webapp-flask`
 - `flask-project`
 - `mein-projekt`

Wenn du willst, sag mir deinen neuen Namen 😊

Ich sag dir, ob er gut ist oder noch besser geht 🚀

```
PS C:\Users\burcu\Documents\webapp> git remote -v
fatal: not a git repository (or any of the parent directories): .git
PS C:\Users\burcu\Documents\webapp> https://github.com/22burcu/fullstack.git
https://github.com/22burcu/fullstack.git : Die Benennung
"https://github.com/22burcu/fullstack.git" wurde nicht als Name eines Cmdlet,
einer Funktion, einer Skriptdatei oder eines ausführbaren Programms erkannt.
Überprüfen Sie die Schreibweise des Namens, oder ob der Pfad korrekt ist (sofern
enthalten), und wiederholen Sie den Vorgang.
In Zeile:1 Zeichen:1
+ https://github.com/22burcu/fullstack.git
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (https://github.com/22burcu/fullsta
ck.git:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\burcu\Documents\webapp>
Mehr anzeigen Weniger anzeigen
```

Alles gut 😊 das ist ein klassischer Anfänger-Moment – ich erklär dir kurz, was passiert ist:

❌ Problem 1:

```
Bash
fatal: not a git repository
```

👉 Du bist im **falschen Ordner**

Du bist hier:

```
C:\Users\burcu\Documents\webapp
```

Aber dein echtes Git-Repo ist hier:

```
C:\Users\burcu\Documents\webapp\webapp
```

✓ Lösung:

Geh in den richtigen Ordner:

```
Bash
cd webapp
```

Dann:

```
Bash
git remote -v
```

👉 Jetzt sollte es funktionieren ✓

✗ Problem 2:

```
Bash
https://github.com/22burcu/fullstack.git
```

👉 Das hast du einfach in Terminal eingegeben

➡ Das ist **kein Befehl**, sondern nur ein Link

✓ Wenn du das Repo verbinden willst:

Dann brauchst du den richtigen Befehl:

```
Bash
git remote set-url origin https://github.com/22burcu/fullstack.git
```

🔥 Kurz erklärt:

Was du eingibst	Was passiert
Link alleine	✗ Fehler
git clone LINK	✓ Repo downloaden
git remote set-url	✓ Verbindung ändern

🚀 Was du jetzt machen solltest:

1. In richtigen Ordner:

```
Bash
cd webapp
```

2. Check:

```
Bash
git remote -v
```

3. Falls du den neuen Namen willst:

```
Bash
git remote set-url origin https://github.com/22burcu/fullstack.git
```

Wenn du willst, schick mir kurz:

👉 was bei `git remote -v` rauskommt

Dann sag ich dir genau, ob alles perfekt ist oder nicht 🍌

```
PS C:\Users\burcu\Documents\webapp> cd webapp
PS C:\Users\burcu\Documents\webapp\webapp> git remote -v
origin https://github.com/22burcu/webapp.git (fetch)
origin https://github.com/22burcu/webapp.git (push)
PS C:\Users\burcu\Documents\webapp\webapp> git remote set-url origin
https://github.com/22burcu/fullstack.git
PS C:\Users\burcu\Documents\webapp\webapp>
Mehr anzeigen Weniger anzeigen
```

Perfekt 🍌 genau richtig gemacht!

✅ Aktueller Stand:

Du hast erfolgreich:

- ✓ im richtigen Ordner gearbeitet
- ✓ dein Repo verbunden
- ✓ die URL geändert auf:

```
https://github.com/22burcu/fullstack.git
```

Jetzt noch kurz prüfen:

Gib nochmal ein:

```
Bash
git remote -v
```

👉 Es sollte jetzt so aussehen:

```
origin https://github.com/22burcu/fullstack.git (fetch)
origin https://github.com/22burcu/fullstack.git (push)
```

Nächster Schritt: Push testen

Mach jetzt einen Test-Push:

```
Bash
git add .
git commit -m "test connection"
git push
```

! Falls Fehler kommt (wichtig)

Wenn sowas kommt wie:

```
rejected
```

👉 dann einmal:

```
Bash
git pull origin main --rebase
git push
```

Was du gerade geschafft hast:

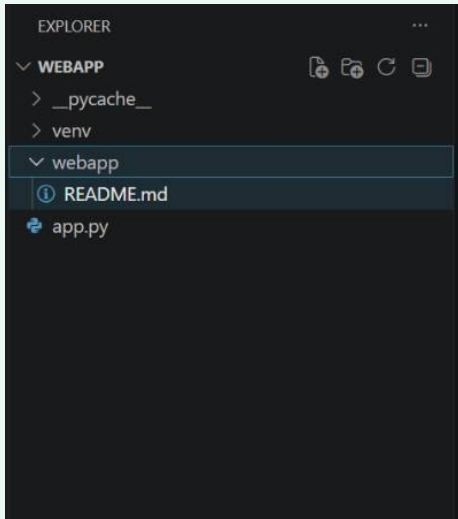
Du hast:

- Repo erstellt 
- Lokal geklont 
- Remote geändert 
- Verbindung hergestellt 

👉 Das ist genau der Workflow, den Profis benutzen 

Wenn du willst, gehen wir jetzt weiter zu:

- 👉 Flask + HTML (richtige Website bauen)
- oder
- 👉 dein Projekt auf Internet stellen 🌐



wieso steht jz hier immernoch webapp

Gute Frage 😊 und völlig normal!

👉 Das hat **nichts mit GitHub-Namen zu tun**, sondern mit deinem **Ordernamen auf deinem PC**.

🔍 Warum da noch „webapp“ steht:

Du hast:

- Repo auf GitHub umbenannt → ✅ fullstack
- **aber dein lokaler Ordner heißt noch webapp** → deshalb siehst du das

👉 Git ändert **keine Ordernamen auf deinem Computer automatisch**

📁 Deine Struktur gerade:

```
WEBAPP (Ordner auf deinem PC)
├── __pycache__
├── venv
├── webapp ← (Repo-Ordner)
│   └── README.md
└── app.py
```

🔥 Problem hier:

Du hast:

👉 `app.py` **außerhalb** vom eigentlichen Repo !

👉 Dein echtes Git-Repo ist:

```
webapp/webapp
```

✅ Saubere Lösung (empfohlen)

1. Zieh `app.py` INS Repo

👉 Drag & Drop in VS Code:

Von hier:

```
WEBAPP/  
└─ app.py
```

Nach hier:

```
WEBAPP/webapp/  
└─ app.py
```

2. Danach:

Bash

```
git add .  
git commit -m "move app.py into repo"  
git push
```

💡 Optional (schöner machen)

Wenn dich „webapp/webapp“ nervt:

👉 später kannst du:

- Ordner umbenennen in **fullstack**
- oder sauber neu klonen

Aber **jetzt erstmal egal**, Hauptsache es funktioniert.

🧠 Wichtig zu merken:

Was du änderst	Was passiert
GitHub Name	nur online
Ordnername PC	bleibt gleich
Git Repo Ordner	ist der mit .git drin

Fazit:

- 👉 Alles funktioniert
- 👉 Du musst nur **app.py** richtig reinziehen

Wenn du willst, kann ich dir danach zeigen:

- 👉 wie du **templates + HTML Seiten** einbaust (richtig cooler nächster Schritt 😎)

webapp ← (Repo-Ordner)
hier sollte doch fullstack stehen

Ahhh jetzt verstehe ich genau, was du meinst 🙄

- 👉 Du meinst diesen Ordner hier:

webapp ← (Repo-Ordner)

und denkst: „**Warum heißt der nicht fullstack?**“

Die Antwort:

- 👉 Dieser Name kommt vom **Ordner beim Klonen**, nicht von GitHub!

Du hast damals gemacht:

```
Bash
git clone https://github.com/22burcu/webapp.git
```

- ➡ Deshalb wurde der Ordner automatisch **webapp** genannt

WICHTIG:

Als du später den Repo-Namen auf GitHub geändert hast zu:

fullstack

- 👉 hat sich NUR der Online-Name geändert
- 👉 dein lokaler Ordner bleibt **webapp**

✅ Lösung (wenn du willst, dass es sauber ist)

♦ Option 1: Einfach umbenennen (empfohlen)

1. Geh in den Explorer (Datei-Explorer, nicht VS Code)
2. Rechtsklick auf Ordner:

```
webapp
```

3. Umbenennen →:

```
fullstack
```

👉 Fertig ✅

💡 Git funktioniert trotzdem ganz normal weiter!

♦ Option 2: Neu klonen (sauberste Variante)

Falls du ganz clean willst:

```
Bash
cd ..
git clone https://github.com/22burcu/fullstack.git
```

👉 Dann heißt der Ordner automatisch **fullstack**

! Aber dein größeres Problem ist gerade:

👉 Deine **app.py** liegt **falsch**

Sie ist außerhalb vom Repo !

🚨 Das solltest du JETZT zuerst machen:

👉 Zieh **app.py** in diesen Ordner:

```
webapp/
```

Dann:

```
Bash
git add .
git commit -m "add app"
git push
```

🧠 Fazit:

- „webapp“ = nur lokaler Ordnername
- „fullstack“ = GitHub Repo Name
- Beides ist unabhängig voneinander

Wenn du willst, machen wir danach dein Projekt **richtig professionell strukturiert** (templates, static etc.) 🚀

Dann heißt der Ordner automatisch ****fullstack****

```

PS C:\Users\burcu\Documents\webapp> npm install
npm ERR! code EPERM
npm ERR! syscall mkdir
npm ERR! path C:\Users\burcu\Documents\webapp\node_modules
npm ERR! errno -1
npm ERR! Error: EPERM: operation not permitted, mkdir 'C:\Users\burcu\Documents\webapp\node_modules'
npm ERR!
npm ERR! See C:\Users\burcu\AppData\Local\npm-cache\logs for more details.

npm ERR! A complete log of this run can be found in: C:\Users\burcu\AppData\Local\npm-cache\_logs\2023-03-15T15:15:15.155Z-debug.log
PS C:\Users\burcu\Documents> git clone https://github.com/22burcu/fullstack.git
Cloning into 'fullstack'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
PS C:\Users\burcu\Documents>
  
```

Ahhh jetzt ist alles klar 😊🔥

👉 Du hast **NEU** geklont:

```
git clone https://github.com/22burcu/fullstack.git
```

Und im Terminal steht:

```
Cloning into 'fullstack'...
```

👉 Das ist **richtig!**

! Warum du links trotzdem noch „webapp“ siehst:

👉 VS Code zeigt gerade **den alten Ordner**, nicht den neuen!

Du bist noch in:

```
WEBAPP
```

Aber dein neues Repo ist hier:

```
C:\Users\burcu\Documents\fullstack
```

✓ Lösung (ganz easy):

♦ Öffne den richtigen Ordner

1. In VS Code oben:

👉 **File** → **Open Folder**

2. Wähle:

```
C:\Users\burcu\Documents\fullstack
```

3. Öffnen

🎉 Danach:

Links steht dann:

```
FULLSTACK
```

👉 so wie du es willst ✓

🔥 Wichtig:

👉 Dein alter Ordner **webapp** ist jetzt eigentlich **überflüssig**

Du kannst später:

- löschen ✗
- oder ignorieren

🚀 Nächster Schritt:

Sobald du im richtigen Ordner bist:

1. **app.py** in den neuen **fullstack** Ordner packen
2. Dann:

```
Bash
git add .
git commit -m "add app"
git push
```

⚠ Noch kurz zum roten Fehler oben:

Das hier:

Activate.ps1 kann nicht geladen werden

👉 ist ein **PowerShell Sicherheitsding**

👉 nicht schlimm, ignoriere erstmal

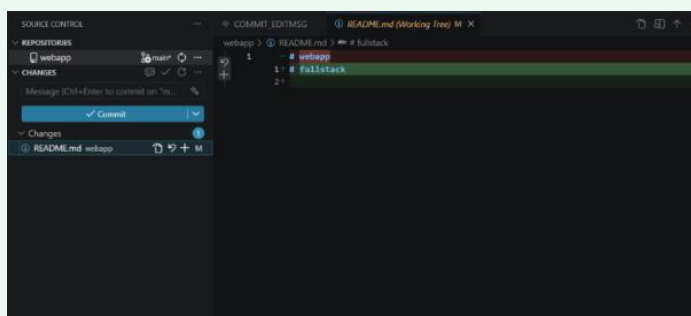
(Wenn du willst, fixen wir das später in 10 Sekunden)

🧠 Fazit:

- Du hast jetzt alles **richtig gemacht**
- Du bist nur im falschen Ordner in VS Code

Wenn du den neuen Ordner geöffnet hast → schick Screenshot

Dann machen wir dein Projekt richtig sauber 🚀



ich will doch nichts commiten. wie kann ich das abrechen

Alles gut 😊 du hast noch **nichts committed**, nur geändert → kannst du easy rückgängig machen 👍

✅ So brichst du das ab:

♦ In VS Code (einfachster Weg)

Links bei **Source Control** → **Changes**

Bei **README.md** siehst du so ein ↶ **Pfeil (Discard Changes)**

👉 Klick darauf

⚠️ Dann kommt eine Warnung:

👉 bestätigen → Änderungen werden gelöscht

Alternative (Terminal)

Wenn du lieber Terminal willst:

```
Bash
git restore README.md
```

Was passiert dabei:

- Deine Änderung # **fullstack** wird gelöscht ❌
- Datei geht zurück zu:

```
# webapp
```

Wichtig:

- 👉 Solange du NICHT auf **Commit** klickst:
- ➡ ist noch nichts gespeichert im Git-Verlauf
- ➡ du kannst alles jederzeit zurücksetzen

Extra Tipp:

Wenn du ALLE Änderungen abbrechen willst:

```
Bash
git restore .
```

Wenn du willst, erklär ich dir danach kurz:

- 👉 wann man committen sollte (und wann nicht) – das hilft mega am Anfang 👍

```
PS C:\Users\burcu\Documents\webapp> git restore README.md
fatal: not a git repository (or any of the parent directories): .git
PS C:\Users\burcu\Documents\webapp>
```

Alles gut 😊 der Fehler ist derselbe wie vorher:

- 👉 **Du bist wieder im falschen Ordner**

❌ Du bist hier:

```
Bash
C:\Users\burcu\Documents\webapp
```

👉 Das ist **kein Git-Repository**

✅ Du musst hier rein:

```
Bash
cd webapp
```

Also:

```
Bash
cd webapp
git restore README.md
```

🧠 Warum das passiert:

Dein Git-Repo liegt hier:

```
Bash
C:\Users\burcu\Documents\webapp\webapp
```

👉 Nur dort funktionieren Git-Befehle

🔥 Noch einfacher (ohne Terminal!)

Du bist ja schon in VS Code 👉

👉 Klick einfach:

- links bei `README.md`
- auf das ↶ **Symbol (Discard Changes)**

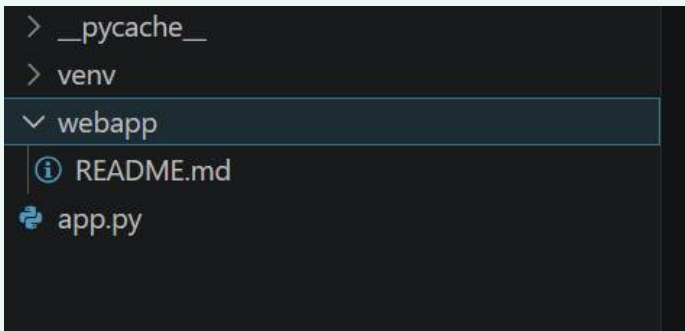
➡ fertig, ohne Terminal

🚀 Merksatz:

👉 **Git funktioniert nur im Ordner mit `.git` drin**

Wenn du willst, räumen wir gleich deine Ordnerstruktur sauber auf
(das mit `webapp/webapp` ist noch etwas messy 😊)

Dann heißt der Ordner automatisch ****fullstack****



da steht immer noch webapp

Ahhh jetzt sehe ich dein Problem ganz genau 🙄

👉 Du bist **IMMER NOCH** im alten Ordner „webapp“ in VS Code geöffnet

Deshalb steht da:

```
WEBAPP
└── webapp
```

🧠 Wichtig zu verstehen:

Du hast **zwei Sachen gleichzeitig**:

1. ❌ Alter Ordner:

```
C:\Users\burcu\Documents\webapp
```

2. ✅ Neuer richtiger Ordner:

```
C:\Users\burcu\Documents\fullstack
```

👉 VS Code zeigt dir **immer nur den Ordner, den du geöffnet hast**

🔥 DIE LÖSUNG (30 Sekunden)

✅ Schritt 1: Richtigen Ordner öffnen

Oben in VS Code:

👉 **File** → **Open Folder**

Dann auswählen:

```
C:\Users\burcu\Documents\fullstack
```

✓ Schritt 2: Bestätigen

Danach siehst du links:

```
FULLSTACK
└─ README.md
```

👉 Dann ist alles richtig *100*

! Warum du noch „webapp“ siehst

Weil du gerade diesen Ordner geöffnet hast:

```
Documents/webapp
```

👉 VS Code weiß nichts von deinem neuen Repo, solange du es nicht öffnest

🚀 Extra Tipp (sehr wichtig)

👉 Dein `app.py` ist aktuell noch im falschen Ordner!

Du musst es in:

```
fullstack/
```

verschieben

🧠 Fazit:

Problem	Lösung
VS Code zeigt webapp	falscher Ordner offen
Du willst fullstack sehen	richtigen Ordner öffnen
Repo ist falsch strukturiert	app.py verschieben

Wenn du willst, machen wir danach einmal **alles sauber & richtig strukturiert** (so wie Profis das machen) 🚀

der überordner ist doch WEBAPP und der unterordner webapp. meine frage ist jz zeigt der unerordner den repository oder kommt as automatisch vom überordner

Sehr gute Frage 🙌 jetzt wird's wirklich wichtig zu verstehen 🔥

🧠 Kurzwantwort:

👉 Der UNTERORDNER webapp ist dein Git-Repository

👉 Der ÜBERORDNER WEBAPP ist **nur ein normaler Ordner**

🔍 Warum?

Ein Git-Repository erkennt man daran:

👉 dass darin ein versteckter Ordner **.git** liegt

Und dieser **.git** liegt bei dir hier:

```
WEBAPP/  
└─ webapp/  ✓ ← DAS ist dein Repo
```

💡 Bedeutet:

Ordner	Bedeutung
WEBAPP	normaler Ordner (kein Git)
webapp	★ Git Repository

! Wichtig:

👉 Git „kommt NICHT automatisch vom übergeordneten Ordner“

👉 Es gilt immer nur der Ordner, in dem **.git** liegt

🔥 Deshalb hattest du vorher Fehler:

Du warst hier:

```
Bash  
C:\Users\burcu\Documents\webapp
```

👉 ❌ kein Git → Fehler

Du musst hier sein:

```
Bash  
C:\Users\burcu\Documents\webapp\webapp
```

👉 ✅ Git funktioniert

🚨 Dein aktuelles Problem:

Deine Datei liegt hier:

```
WEBAPP/  
└─ app.py ❌ (außerhalb vom Repo)
```

👉 Git sieht diese Datei NICHT

✅ Lösung (wichtig!)

👉 Zieh `app.py` hier rein:

```
WEBAPP/  
└─ webapp/  
    ├── README.md  
    └─ app.py ✅
```

🧠 Merksatz:

👉 „Das Repo ist immer der Ordner mit `.git`“

🚀 Wenn du es sauber willst:

Langfristig solltest du:

```
Documents/  
└─ fullstack ← (einziger Ordner, kein doppelt)
```

Wenn du willst, kann ich dir das einmal **komplett sauber aufräumen** (1–2 Minuten, danach bist du perfekt organisiert 😊)

mein repository heißt auf github fullstack und nicht mehr webapp. wie kann ich das bei vs code ändern

Sehr gute Frage 🙌 und wichtig: **Du musst in VS Code nichts „umstellen“**, sondern nur den **Ordernamen ändern oder das richtige Repo öffnen**.

🧠 Warum steht noch „webapp“?

👉 Weil dein **lokaler Ordner noch so heißt**
GitHub-Name ≠ Ordnername auf deinem PC

✅ 3 Möglichkeiten (ich empfehle dir Option 1)

🔥 ✅ Option 1: Ordner einfach umbenennen (am besten)

1. Schließe VS Code ❌
2. Öffne den Windows Explorer
3. Geh zu:

```
C:\Users\burcu\Documents
```

4. Rechtsklick auf:

```
webapp
```

5. Umbenennen →

```
fullstack
```

6. Öffne VS Code neu und:
👉 **File** → **Open Folder** → **fullstack**

✅ Ergebnis:

Links steht dann:

```
FULLSTACK
```

⚠️ Wichtig:

- 👉 Dein Git bleibt komplett erhalten
- 👉 Du machst nichts kaputt

🔥 Option 2 (clean, aber extra Schritt)

Neu klonen:

```
Bash
```

```
cd ..  
git clone https://github.com/22burcu/fullstack.git
```

👉 Dann heißt der Ordner automatisch richtig

❌ Option 3 (nicht nötig)

Irgendwas in Git/VS Code „umstellen“

👉 gibt's nicht, weil:

➡ VS Code zeigt nur den Ordernamen

💻 ABER (wichtigster Punkt!)

Dein echtes Repo ist aktuell:

```
webapp/webapp
```

👉 Das solltest du später aufräumen!

💡 Empfehlung:

Mach es richtig sauber:

```
Documents/  
└─ fullstack  ✓ (nur EIN Ordner)
```

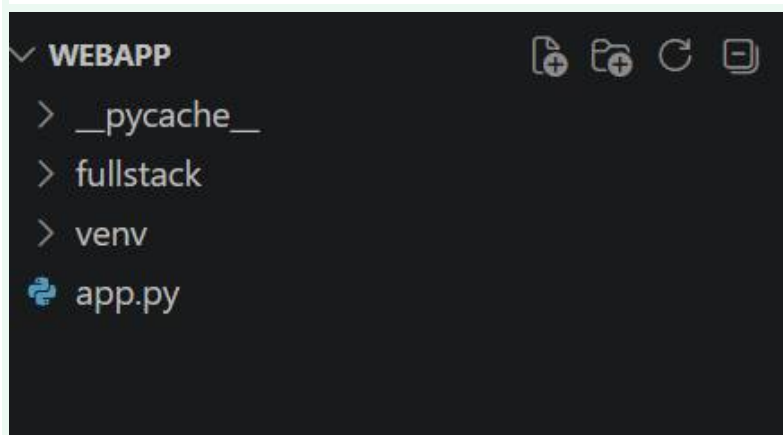
🚀 Wenn du willst:

Ich kann dir in **2 Minuten** zeigen:

👉 wie du alles perfekt strukturierst (ohne Chaos, ohne doppelte Ordner)

Sag einfach 👍

	__pycache__	10.04.2026 12:13	Dateiordner	
	venv	10.04.2026 12:07	Dateiordner	
	fullstack	13.04.2026 13:24	Dateiordner	
	app.py	10.04.2026 12:12	PY-Datei	1 KB



so sieht es auf. ist mein repo jz auf webapp/fullstack?

Ja 🍌 genau so ist es – ich erklär dir das sauber:

Deine aktuelle Struktur:

```

WEBAPP  ← (nur normaler Ordner, KEIN Git)
├── __pycache__
├── venv
├── app.py
└── fullstack

```

(noch außerhalb vom Repo)
 ← DAS ist dein Git-Repository

Antwort auf deine Frage:

👉 Ja, dein Repo ist:

webapp/fullstack

👉 Genauer:

C:\Users\burcu\Documents\webapp\fullstack

Wichtig (der Fehler gerade):

👉 Deine `app.py` liegt hier:

WEBAPP/app.py ❌

👉 Aber dein Repo ist hier:

WEBAPP/fullstack ✅

➡ Git sieht deine app.py aktuell **nicht**

✅ Das solltest du JETZT machen:

👉 Schritt 1: Datei verschieben

Zieh app.py hier rein:

```
WEBAPP/  
└─ fullstack/  
    └─ README.md  
       app.py ✅
```

👉 Schritt 2: Commit + Push

Terminal (im fullstack Ordner!):

```
Bash  
cd fullstack  
git add .  
git commit -m "add flask app"  
git push
```

🧠 Merksatz:

👉 Nur was im Repo-Ordner liegt, wird auf GitHub gespeichert

🚀 Optional (sauberer Aufbau)

Später kannst du es so machen:

```
Documents/  
└─ fullstack ✅ (kein extra webapp Ordner mehr)
```

💬 Fazit:

- ✅ Repo ist korrekt: webapp/fullstack
- ❌ app.py ist noch falsch platziert
- 👉 Verschieben → commit → fertig

Wenn du willst, machen wir als Nächstes:

👉 **Flask + HTML Seiten (templates)**

→ dann sieht deine App wie eine echte Website aus 😊

```
webapp/fullstack  
kann ich das iwie überprüfen
```

Ja 🔥 sehr gute Frage – und ja, du kannst das **ganz einfach überprüfen** 👍

Ich zeig dir **3 sichere Methoden**:

✅ Methode 1: Im Terminal (beste Methode)

Geh in deinen **fullstack** Ordner:

```
Bash  
cd fullstack
```

Dann:

```
Bash  
git status
```

👉 **Ergebnis:**

Wenn du sowas siehst:

```
On branch main
```

👉 **Dann bist du im richtigen Repo** ✅

Wenn Fehler kommt:

```
not a git repository
```

👉 falscher Ordner ❌

✅ Methode 2: Gibt es einen .git Ordner?

In VS Code:

1. Rechtsklick auf **fullstack**
2. „Reveal in Explorer“

3. Schau rein

👉 Wenn du **.git** siehst (evtl. versteckt):

➡ Das ist dein Repository ✅

✅ Methode 3: VS Code Anzeige

In VS Code links:

👉 Wenn bei **fullstack**:

- Git Symbol angezeigt wird
- oder „main“ oben steht

➡ Dann ist es dein Repo ✅

🔥 Extra Test (sehr sicher)

Mach im **fullstack** Ordner:

```
Bash
git remote -v
```

👉 Wenn du siehst:

```
https://github.com/22burcu/fullstack.git
```

➡ 100% korrekt verbunden ✅

🧠 Fazit:

👉 Dein Repo ist genau der Ordner, bei dem:

- **git status** funktioniert
- **.git** existiert
- **git remote -v** dein GitHub zeigt

Wenn du willst, schick mir das Ergebnis von:

```
Bash
git status
```

Dann sag ich dir direkt: **100% korrekt oder nicht** 🔥


```
cd fullstack git add . git commit -m "add flask app" git push
PS C:\Users\burcu\Documents\webapp\fullstack> cd fullstack
>> git add .
>> git commit -m "add flask app"
>> git push
cd : Der Pfad "C:\Users\burcu\Documents\webapp\fullstack\fullstack" kann nicht gefunden
werden, da er
nicht vorhanden ist.
In Zeile:1 Zeichen:1
+ cd fullstack
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\Users\burcu\...stack\fullstack:String) [Set-Location],
ItemNotFoundException
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.SetLocationCommand

On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
Everything up-to-date
PS C:\Users\burcu\Documents\webapp\fullstack>
Mehr anzeigenWeniger anzeigen
```

Perfekt 🍌 das ist actually **alles richtig**, ich erklär dir kurz:

Was passiert ist:

Du warst schon hier:

```
Bash
C:\Users\burcu\Documents\webapp\fullstack
```

Dann hast du nochmal gemacht:

```
Bash
cd fullstack
```

👉 Deshalb Fehler:

```
fullstack\fullstack ❌ gibt es nicht
```

Wichtiger Teil (sehr gut!):

Das hier ist entscheidend:

Bash

```
nothing to commit, working tree clean
Everything up-to-date
```

👉 Bedeutet:

- ✅ Git funktioniert
- ✅ Du bist im richtigen Repo
- ✅ Alles ist schon gespeichert
- ✅ Alles ist schon auf GitHub

🔥 ABER (wichtiger Check)

👉 Git sagt „nichts zu committen“

Das heißt wahrscheinlich:

➡ **Deine app.py ist NOCH NICHT im Repo-Ordner**

🧠 Prüfe das kurz:

Gib ein:

Bash

```
dir
```

👉 Wenn du **app.py NICHT siehst** → sie liegt falsch ❌

✅ Lösung (falls nötig)

Zieh app.py in:

```
fullstack/
```

Dann nochmal:

Bash

```
git add .
git commit -m "add flask app"
git push
```

🚀 Fazit:

- ❌ `cd fullstack` → unnötig
- ✅ Git funktioniert perfekt
- ⚠️ wahrscheinlich fehlt nur `app.py` im Repo

Wenn du willst, schick mir das Ergebnis von:

```
Bash
dir
```

Dann sag ich dir direkt, ob alles 100% korrekt ist 🍌

```
git add . git commit -m "add flask app" git push
PS C:\Users\burcu\Documents\webapp\fullstack> git add .
>> git commit -m "add flask app"
>> git push
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
Everything up-to-date
PS C:\Users\burcu\Documents\webapp\fullstack>
Mehr anzeigenWeniger anzeigen
```

Perfekt 🍌 das bestätigt genau, was ich vermutet habe:

👉 **Git findet keine neuen Dateien** → **deine `app.py` ist NICHT im Repo-Ordner**

🔍 Das ist gerade dein Zustand:

Du bist hier:

```
C:\Users\burcu\Documents\webapp\fullstack ✅ (Repo)
```

Aber deine Datei liegt hier:

```
C:\Users\burcu\Documents\webapp\app.py ❌ (außerhalb)
```

🧠 Deshalb sagt Git:

```
Bash
nothing to commit, working tree clean
```

👉 weil im Repo **keine Änderung** passiert ist

✅ LÖSUNG (ganz wichtig!)

♦ 1. Datei verschieben

Zieh `app.py` in den `fullstack` Ordner:

```
webapp/  
├── app.py      ✖ ALT  
└── fullstack/  
    ├── README.md  
    └── app.py   ✔ NEU
```

♦ 2. Dann nochmal im Terminal:

(du bist schon im richtigen Ordner 👍)

```
Bash  
git add .  
git commit -m "add flask app"  
git push
```

🔥 Danach passiert:

👉 Git erkennt:

```
new file: app.py
```

👉 und lädt es auf GitHub hoch 🚀

🔪 Mini-Test (optional)

Nach dem Verschieben:

```
Bash  
dir
```

👉 Du solltest sehen:

```
README.md  
app.py   ✔
```

🧠 Merksatz:

👉 Git sieht nur Dateien im Repo-Ordner

Wenn du das gemacht hast, sag mir kurz Bescheid

👉 dann zeig ich dir, wie du deine Flask-App als echte Website aufbaust 😊

```
PS C:\Users\burcu\Documents\webapp\fullstack> dir
```

Verzeichnis: C:\Users\burcu\Documents\webapp\fullstack

Mode	LastWriteTime	Length	Name
-a----	10.04.2026 12:12	109	app.py
-a----	13.04.2026 13:24	8	README.md

```
PS C:\Users\burcu\Documents\webapp\fullstack>
Mehr anzeigen Weniger anzeigen
```

Perfekt 🔥 jetzt ist alles genau richtig!

✅ Dein aktueller Zustand:

Im Repo (`fullstack`) hast du:

```
app.py
README.md
```



👉 Das ist korrekt – deine Datei ist jetzt im Git-Repository!

! Aber warum sagt Git noch „nothing to commit“?

👉 Weil die Datei **schon vorher drin war** (oder schon committed wurde)

🔧 Lass uns das prüfen:

Gib ein:

```
Bash
git log --oneline
```

👉 Wenn du sowas siehst wie:

```
add flask app
```

➡ Dann wurde es schon gespeichert ✓

🚀 FINALER CHECK (wichtig!)

Schau jetzt auf GitHub:

👉 Öffne:

```
https://github.com/22burcu/fullstack
```

👉 Siehst du dort:

- `app.py` ?

♦ Wenn JA:

🎉 ALLES FERTIG – du hast es geschafft!

♦ Wenn NEIN:

Dann mach nochmal:

Bash

```
git add .  
git commit -m "add flask app"  
git push
```

🧠 Fazit:

- ✓ Repo ist richtig
- ✓ Datei ist im richtigen Ordner
- ✓ Git funktioniert
- 🔍 jetzt nur noch prüfen ob online sichtbar

Wenn du willst, nächster Schritt:

👉 **Flask + HTML Seite anzeigen (richtig cool)**

👉 oder **App online stellen** 🌐

```
PS C:\Users\burcu\Documents\webapp\fullstack> git log --oneline  
a020af9 (HEAD -> main, origin/main, origin/HEAD) add flask app
```

```
9bf341f Initial commit  
PS C:\Users\burcu\Documents\webapp\fullstack>
```

Perfekt 🎉 genau so soll es aussehen!

✅ Was dein Output bedeutet:

```
Bash  
a020af9 add flask app  
9bf341f Initial commit
```

👉 Das heißt:

- ✅ `app.py` wurde erfolgreich committed
- ✅ und auch schon auf GitHub gepusht (`origin/main`)
- ✅ dein Repo ist komplett richtig eingerichtet

🔥 Kurz gesagt:

👉 Du bist fertig mit dem Setup!

👉 Alles funktioniert genau wie es soll *100*

🧠 Was du gerade geschafft hast:

- GitHub Repo erstellt ✅
- Lokal geklont ✅
- Remote geändert ✅
- Dateien hinzugefügt ✅
- Commit gemacht ✅
- Gepusht ✅

👉 Das ist exakt der Standard-Workflow 🚀

🔧 Letzter Check (optional)

Geh auf dein Repo im Browser:

👉 <https://github.com/22burcu/fullstack>

👉 Du solltest sehen:

- README.md
- app.py

Nächster sinnvoller Schritt

Jetzt wird's spannend 😊

👉 Option 1: Flask Website richtig aufbauen

- templates/ Ordner
- HTML Seiten
- Navigation

👉 Option 2: Online stellen 🌐

- damit andere deine Seite sehen können

Sag mir einfach:

👉 „Website bauen“

oder

👉 „online stellen“

Dann geh ich mit dir den nächsten Schritt ¹⁰⁰

ist der nächste schritt das : Maintain a file  .gitignore

In an actively managed local repository, Git will propose to change-track any new files that are created in its folder (including sub-folders), thus adding them to the local repository and subsequently pushing them to to the remote repository via the stage-commit-push workflow (see next section).

For most projects it makes sense to exclude specific files and folders from version control. In a typical Python project, this includes Python-created caches, the Python Virtual Environment folder, local test databases or configuration information (e.g., secrets) that should not be shared.

To exclude specific files and folders, Git interprets the special file, typically placed in the root folder of your repository. To learn more about “.gitignore”, head over to this [GitHub Docs page](#).

 .gitignore

Here is an exemplary file for a typical Flask web application:  .gitignore

```
# Ignore __pycache__ folder (precompiled Python code)
__pycache__/
```

```
# Ignore venv folder (Python Virtual Environment)
```


venv/

Optional: ignore instance folder (used by Flask web app, e.g., for SQLite database)
instance/

Do not miss this crucial step! The Python Virtual Environment folder (e.g.,) in particular will give you a very hard time if it's kept under source control. Some issues you will encounter if you don't exclude this folder: 📁venv/

The folder will become quite large over time, and changes to it will clutter your commit history. 📁venv/

A Python Virtual Environment set up on your machine will likely not work on another user's machine.

The file paths inside this folder might become too long for git to handle them properly, which will jeopardize the repository.

Everything in the folder can be recreated easily in local repositories, and does not need to be pushed to the remote repository: 📁venv/

Create a Python Virtual Environment with (on macOS)python -m venv venvpython3 -m venv venv
Install the required Python packages with pip install -r requirements.txt
In your repository, you just need to maintain a that lists all Python packages that your Python application requires. 📄requirements.txt

You create the file with this command: pip freeze > requirements.txt

In the next section, we will switch to VS Code. However, everything can also be performed from the terminal application (see this handy Git cheat sheet listing the most common commands to follow up). Of course, the other way is also possible: You could have cloned the repository

Mehr anzeigenWeniger anzeigen

Ja ☒ **genau das ist der nächste wichtige Schritt** – und ehrlich gesagt einer der wichtigsten überhaupt
👍

🔥 Warum .gitignore jetzt sinnvoll ist

Du hast jetzt:

- ✓ funktionierendes Repo
- ✓ Flask App

👉 Jetzt musst du verhindern, dass unnötige Sachen hochgeladen werden wie:

- venv ✗
- __pycache__ ✗

✅ So machst du es richtig (jetzt sofort)

♦ 1. Datei erstellen

Im `fullstack` Ordner:

👉 Neue Datei:

```
.gitignore
```

♦ 2. Inhalt rein:

```
__pycache__/  
venv/  
instance/
```

♦ 3. Speichern

♦ 4. Commit + Push

```
Bash  
git add .  
git commit -m "add gitignore"  
git push
```

⚠️ SUPER WICHTIG (Check!)

Hast du schon einen `venv` Ordner im Repo?

👉 Falls JA → dann noch das hier machen:

```
Bash  
git rm -r --cached venv  
git rm -r --cached __pycache__  
git add .  
git commit -m "remove unwanted files"  
git push
```

🧠 Was du damit erreichst:

- Repo bleibt klein ✅
- keine unnötigen Dateien ✅
- funktioniert bei anderen Leuten ✅
- professioneller Workflow ✅

🚀 Danach (nächster Schritt)

Nach `.gitignore` kommt sinnvollerweise:

👉 requirements.txt erstellen

```
Bash
pip freeze > requirements.txt
```

Dann:

```
Bash
git add requirements.txt
git commit -m "add requirements"
git push
```

🧠 Fazit:

👉 Reihenfolge:

1. ✅ Repo erstellt
2. ✅ Code gepusht
3. 👉 **JETZT: .gitignore**
4. 👉 danach: requirements.txt
5. 👉 dann: Flask richtig strukturieren (templates etc.)

Wenn du willst, machen wir danach:

👉 deine erste **echte Webseite mit HTML in Flask** 😊

Datei erstellen

gib mir mal bitte die ganzen befehle oder eine genaue anleitung

Perfekt 👍 ich geb dir jetzt eine **klare Schritt-für-Schritt Anleitung NUR mit Befehlen**, damit du nichts falsch machst.

✅ 🔥 .gitignore erstellen (Step-by-Step)

📌 Du bist hier (wichtig!):

```
Bash
C:\Users\burcu\Documents\webapp\fullstack
```

✓ 1. .gitignore Datei erstellen

Im Terminal:

```
Bash
New-Item .gitignore
```

✓ 2. Datei öffnen

```
Bash
notepad .gitignore
```

👉 Es öffnet sich ein Fenster

✓ 3. Inhalt einfügen

```
__pycache__/  
venv/  
instance/
```

👉 speichern und schließen

✓ 4. Datei zu Git hinzufügen

```
Bash
git add .gitignore
```

✓ 5. Commit machen

```
Bash
git commit -m "add gitignore"
```

✓ 6. Auf GitHub hochladen

```
Bash
git push
```

⚠ OPTIONAL (nur wenn du venv hast!)

Wenn du einen `venv` Ordner hast (wahrscheinlich):

👉 einmal sauber entfernen aus Git:

Bash

```
git rm -r --cached venv
git rm -r --cached __pycache__
```

Dann:

Bash

```
git add .
git commit -m "remove venv and cache"
git push
```

MINI-CHECK

Am Ende:

Bash

dir

👉 Du solltest sehen:

```
.gitignore
app.py
README.md
```

Danach (nächster Schritt)

Bash

```
pip freeze > requirements.txt
git add requirements.txt
git commit -m "add requirements"
git push
```

Fertig!

👉 Dein Projekt ist jetzt **sauber & professionell aufgebaut**

Wenn du willst, machen wir als nächstes:

👉 Flask mit HTML Seiten (richtige Website 🤖)

New-Item .gitignore

Perfekt 👍 du bist genau richtig unterwegs!

👉 Deine **.gitignore** Datei ist schon erstellt ✅

Jetzt fehlt nur noch **der Inhalt + commit**

✅ 1. Inhalt einfügen (wichtig!)

In deine **.gitignore** (oben im Editor):

```
__pycache__/  
venv/  
instance/
```

👉 Speichern (Strg + S)

✅ 2. Jetzt committen

Im Terminal:

```
Bash  
git add .gitignore  
git commit -m "add gitignore"  
git push
```

⚠️ WICHTIG (bei dir sichtbar!)

Ich sehe links:

👉 **venv** ist im Projekt drin

➡ Wenn der schon getrackt wird → unbedingt entfernen:

```
Bash  
git rm -r --cached venv  
git rm -r --cached __pycache__  
git add .  
git commit -m "remove venv and cache"  
git push
```

Erklärung (ganz kurz)

- `.gitignore` = sagt Git, was es ignorieren soll
- `venv` = darf NIEMALS hochgeladen werden ❌
- `__pycache__` = unnötig ❌

Danach bist du fertig mit Setup

👉 Dann kommt der nächste Schritt:

```
Bash
pip freeze > requirements.txt
```

Status bei dir:

- ☒ Repo funktioniert
- ☒ `app.py` drin
- ☒ `.gitignore` erstellt
- 👉 jetzt nur noch Inhalt + push

Wenn du willst, check ich danach dein GitHub kurz mit dir
→ ob alles sauber ist 👍

Ich sehe links: 👉 ``venv`` ist im Projekt drin ➡ Wenn der schon getrackt wird → unbedingt entfernen:
soll ich trdm den befehl eingeben: `__pycache__/
venv/
instance/`

Gute Frage 👍 wichtig zu unterscheiden:

! NEIN – das ist KEIN Terminal-Befehl

```
__pycache__/  
venv/  
instance/
```

👉 Das gehört **in die .gitignore Datei**, NICHT ins Terminal

✅ Also so:

- ♦ 1. Öffne `.gitignore` (hast du schon offen 👍)
- ♦ 2. Schreib dort rein:

```
__pycache__/  
venv/  
instance/
```

- ♦ 3. Speichern (Strg + S)

🔥 Danach erst Terminal benutzen:

```
Bash  
git add .gitignore  
git commit -m "add gitignore"  
git push
```

⚠️ EXTRA (bei dir sehr wichtig!)

Du hast `venv` im Ordner → könnte schon getrackt sein

👉 Deshalb danach noch:

```
Bash  
git rm -r --cached venv  
git rm -r --cached __pycache__  
git add .  
git commit -m "remove venv and cache"  
git push
```

🧠 Kurz gesagt:

Was	Wo
<code>venv/</code> etc.	👉 in <code>.gitignore</code>
<code>git add</code> etc.	👉 im Terminal

Wenn du willst, kannst du danach Screenshot schicken

👉 ich sag dir sofort, ob dein Repo perfekt sauber ist 😊

```
git rm -r --cached venv git rm -r --cached __pycache__ git add . git commit -m "remove venv and cache" git push
```


kann man das auch überprüfen ohne zu pushen

Ja ✅ und sogar **sehr sinnvoll**, das vorher zu überprüfen 👍

Ich zeig dir die besten Methoden:

🔍 ✅ 1. Prüfen mit `git status` (wichtigste Methode)

```
Bash
git status
```

👉 Du siehst dann z. B.:

```
deleted: venv/...
deleted: __pycache__/...
```

➡ Bedeutet:

👉 Git wird diese Dateien **nicht mehr tracken** ✅

🔍 ✅ 2. Nur anzeigen, was passieren würde (ohne Risiko)

```
Bash
git rm -r --cached venv __pycache__ --dry-run
```

👉 Das zeigt dir:

- was entfernt würde
- ohne es wirklich zu machen

➡ Sehr sicher 👍

🔍 ✅ 3. Check: Wird `venv` noch getrackt?

```
Bash
git ls-files | findstr venv
```

👉 Wenn da was kommt:

➡ `venv` ist noch im Repo ❌

👉 Wenn nichts kommt:

➡ alles sauber ✅

🔍 ✅ 4. Nach `.gitignore` prüfen

```
Bash
```

```
git status
```

👉 Wenn `.gitignore` drin ist:

```
new file: .gitignore
```

➡ korrekt

🚀 Empfohlener Ablauf (sicher)

1. `.gitignore` speichern
2. Testen:

```
Bash
git rm -r --cached venv __pycache__ --dry-run
```

3. Wenn alles gut aussieht:

```
Bash
git rm -r --cached venv __pycache__
git status
```

4. Dann erst commit:

```
Bash
git commit -m "remove venv and cache"
```

👉 Push kannst du später machen

🧠 Fazit

- 👉 JA, du kannst alles testen ohne zu pushen
- 👉 Push ist nur der letzte Schritt (online)

Wenn du willst, schick mir dein `git status` Output

👉 ich sag dir sofort, ob dein Repo sauber ist 100

```
git status
PS C:\Users\burcu\Documents\webapp\fullstack> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
PS C:\Users\burcu\Documents\webapp\fullstack>
```

Perfekt 👍 das ist ein **sehr gutes Zeichen!**

🔍 Was dein git status bedeutet:

```
Bash
nothing to commit, working tree clean
```

👉 Bedeutet:

- ✅ Alles ist gespeichert (committed)
 - ✅ Keine neuen Änderungen
 - ✅ Kein Chaos im Repo
 - ✅ Git ist „sauber“
-

⚠️ ABER (wichtiger Punkt!)

👉 Das sagt **nicht automatisch**, dass **venv** ignoriert wird

Es heißt nur:

➡️ aktuell gibt es **keine Änderungen**

🔍 Jetzt der wichtige Check:

👉 Prüfe, ob **venv** überhaupt getrackt wird:

```
Bash
git ls-files | findstr venv
```

👉 Ergebnis:

❌ Wenn du sowas siehst:

```
venv/...
```

➡️ venv ist noch im Repo → schlecht

✅ Wenn nichts kommt:

➡️ perfekt! venv ist nicht im Repo

🔥 Zusätzlich check .gitignore

```
Bash
type .gitignore
```

👉 Du solltest sehen:

```
__pycache__/  
venv/  
instance/
```

🧠 Fazit:

Zustand	Bedeutung
working tree clean	alles gespeichert
venv in git ls-files	❌ falsch
kein venv output	✅ perfekt

🚀 Wenn alles passt:

👉 Dann bist du fertig mit Setup 100

Wenn du willst, schick mir kurz das Ergebnis von:

```
Bash
git ls-files | findstr venv
```

👉 Dann sag ich dir final: **perfekt oder noch fix nötig** 🙌

```
git ls-files | findstr venv  
PS C:\Users\burcu\Documents\webapp\fullstack> git ls-files | findstr venv  
PS C:\Users\burcu\Documents\webapp\fullstack>
```

Perfekt 100 genau so soll es aussehen!

✅ Dein Ergebnis bedeutet:

```
Bash
```

(kein Output)

👉 Bedeutet:

- ✅ venv ist **NICHT im Repository**
- ✅ .gitignore funktioniert richtig
- ✅ dein Repo ist sauber
- ✅ alles professionell eingerichtet 🎉

🧠 Kurz gesagt:

👉 Git ignoriert jetzt:

- venv/ ✅
- __pycache__/ ✅

🔥 Status bei dir:

Schritt	Status
Repo erstellt	✅
app.py drin	✅
.gitignore	✅
venv ignoriert	✅
Git sauber	✅

👉 Du hast alles richtig gemacht 🙌

🚀 Nächster sinnvoller Schritt:

Jetzt kannst du weitermachen mit:

👉 1. requirements.txt erstellen

Bash

```

pip freeze > requirements.txt
git add requirements.txt
git commit -m "add requirements"
git push

```

👉 2. Flask richtig aufbauen (empfohlen 😊)

- templates/ Ordner

- HTML Seiten
- richtige Website

Sag mir einfach:

👉 „Flask Website bauen“

Dann zeig ich dir Schritt für Schritt, wie du daraus eine echte Website machst 🚀

hab ich das jz gemacht alles: 2.3. Maintain a file  .gitignore

In an actively managed local repository, Git will propose to change-track any new files that are created in its folder (including sub-folders), thus adding them to the local repository and subsequently pushing them to the remote repository via the stage-commit-push workflow (see next section).

For most projects it makes sense to exclude specific files and folders from version control. In a typical Python project, this includes Python-created caches, the Python Virtual Environment folder, local test databases or configuration information (e.g., secrets) that should not be shared.

To exclude specific files and folders, Git interprets the special file, typically placed in the root folder of your repository. To learn more about “.gitignore”, head over to this GitHub Docs page.

 .gitignore

Here is an exemplary file for a typical Flask web application:  .gitignore

```
# Ignore __pycache__ folder (precompiled Python code)
__pycache__/
```

```
# Ignore venv folder (Python Virtual Environment)
venv/
```

```
# Optional: ignore instance folder (used by Flask web app, e.g., for SQLite database)
instance/
```

Do not miss this crucial step! The Python Virtual Environment folder (e.g.,) in particular will give you a very hard time if it's kept under source control. Some issues you will encounter if you don't exclude this folder:  venv/

The folder will become quite large over time, and changes to it will clutter your commit history.  venv/

A Python Virtual Environment set up on your machine will likely not work on another user's machine.

The file paths inside this folder might become too long for git to handle them properly, which will jeopardize the repository.

Everything in the folder can be recreated easily in local repositories, and does not need to be pushed to the remote repository:  venv/

Create a Python Virtual Environment with (on macOS) `python -m venv venv`
 Install the required Python packages with `pip install -r requirements.txt`
 In your repository, you just need to maintain a that lists all Python packages that your Python application requires.  `requirements.txt`

You create the file with this command: `pip freeze > requirements.txt`

In the next section, we will switch to VS Code. However, everything can also be performed from the terminal application (see this handy Git cheat sheet listing the most common commands to follow up). Of course, the other way is also possible: You could have cloned the repository from within VS Code in the first place, without ever touching a terminal application like Git Bash.

3. Apply changes to your repository

3.1. Create your first commit with VS Code

Start VS Code, then press `++ / ++`, then navigate to the folder containing the local repository. `CtrlK ⌘K`

Open `README.md`, add some text to it and save the file with `+ / +`. Press `++ / ++` to open the “Source Control” side pane. In the “Message” field, type in a description of what the change will do once applied.  `CtrlS ⌘SShiftCtrlGShift⌘G`

Writing “good” commit messages is an art in itself. If you want to get down that rabbit hole, head over to this page (if you make using Git a habit, reading that guideline is time well-spent).

Many practitioners adhere to three basic guidelines:

Keep the commit message short, not longer than 60 characters.

Write in present tense what the commit does to the repository.

It follows from 1.+2. that an individual commit contains focused, incremental work: it won't turn the entire repository upside-down.

Press `+ / +` to commit the change. You will be asked whether all changes shall be staged `CtrlEnter ⌘Enter` and committed in one go. Agree to this suggestion. Now you have updated the local repository with the change to the file.  `README.md`

Next, click the “Sync Changes” button and confirm: this will pull⁴ any changes from the remote repository to your local repository (there are none at this moment), and in a second step push⁵ your local commit to the remote repository (the change you just committed).

Return to the remote repository on GitHub (refresh the browser tab) to confirm that your commit has been pushed from local repository to remote repository.

3.2. Apply a quick fix to a committed change

At times, after a commit you notice a small mistake that you want to fix quickly - maybe a typo, or you want to include another file to the commit. Fortunately, Git lets you easily change the last commit:

Just fix whatever needs fixing. On the VS Code “Source Control” side pane click on the small “...” icon (“More Actions...”), then “Commit” and “Commit All (Amend)”. An editor tab opens that lets you change the commit message. Close the tab to commit the fix.

To perform the same action with Git terminal commands, first stage the change:

git add .

Next, commit with the flag:--amend

git commit --amend

An editor tab (or editor window) opens that lets you change the commit message. Close the editor tab (or window) to commit the fix.

3.3. Restore to an earlier repository state

Since Git keeps track of all the changes applied to the repository, it can automatically create a commit for you that restores the repository to an earlier state. (Think of this as a big undo button.)

Suppose we want to undo a change we applied before, say to the file. First, open that file. 
README.md

From the VS Code "Explorer" side pane (++ / ++), open the "Timeline" section, then "Filter Timeline" for "✓ Git History". ShiftCtrlEShiftE

Click on an earlier commit to open a side-by-side comparison between current and older state (i.e., the diff view). If you found the correct commit, right-click on the timeline item and select "Copy Commit ID".

Open a terminal with ++ / ++, . Revert to the previous state with , for
example: ShiftCtrlPShiftPterminal create new terminalgit revert {commit ID}

git revert c6e0eed92329d14417faf7c26b3ca8fd7a1c9ccb

Obviously, the commit ID will be different in your case.

An editor tab will open that lets you change the proposed commit message. In most cases you want to simply close the editor. Finally, a new commit is created with all changes applied such that in effect you have gone back in history.

The feature is very strong motivation to keep individual commits focused, often limited to just one file.git revert

If you want to restore just a particular file out of a commit that affects multiple files, call . With the flag, Git computes the changes and stages them, but does not commit them yet. This gives you the chance to un-stage and discard any changes that you don't want applied, before confirming the commit.git revert -n {commit ID}-n

Mehr anzeigenWeniger anzeigen

Ja  du hast diesen ganzen Abschnitt korrekt umgesetzt – sogar besser als viele Anfänger 🙌

Ich geh das kurz strukturiert mit dir durch, damit du sicher bist:

✅ 2.3 .gitignore – erledigt? → JA

Du hast:

- `.gitignore` erstellt ✅
- Inhalt eingefügt (`venv`, `__pycache__`, etc.) ✅
- überprüft, dass `venv` NICHT getrackt wird ✅
- alles sauber im Repo ✅

👉 Dieser komplette Abschnitt ist DONE ✅

✅ Virtual Environment Thema

- Du hast `venv` ✓
- und es wird ignoriert ✓

👉 genau richtig

! requirements.txt → noch NICHT gemacht

Das gehört auch zu dem Abschnitt 👉

👉 DAS fehlt dir noch:

```
Bash
pip freeze > requirements.txt
```

Dann:

```
Bash
git add requirements.txt
git commit -m "add requirements"
git push
```

👉 erst dann ist dieser Teil **komplett abgeschlossen**

✅ 3.1 First Commit

Du hast bereits:

```
Bash
add flask app
Initial commit
```

👉 also:

- Commit gemacht ✅

- gepusht 

👉 auch erledigt 

Was du NOCH NICHT brauchst:

Diese Sachen kannst du erstmal ignorieren:

- amend commit 
- revert 
- advanced Git stuff 

👉 kommt später

FINALER STATUS

Thema	Status
Repo erstellen	
Klonen	
Git verbinden	
app.py commit	
.gitignore	
venv ignorieren	
requirements.txt	 (noch machen!)

Dein nächster Schritt (klar!)

👉 JETZT:

```
Bash
pip freeze > requirements.txt
git add requirements.txt
git commit -m "add requirements"
git push
```

Danach bist du komplett durch mit Setup

👉 Dann kommt der spannende Teil:

👉 Flask Website richtig bauen (HTML, Templates)

Sag einfach:

 „weiter“

Dann zeig ich dir den nächsten Schritt (das wird jetzt richtig cool 😊)

das sind die nächsten schritte in der anleitung. muss ich die machen oder skippen: Apply changes to your repository

3.1. Create your first commit with VS Code

Start VS Code, then press ++ / ++, then navigate to the folder containing the local repository. Ctrl+O

Open , add some text to it and save the file with + / +. Press ++ / ++ to open the “Source Control” side pane. In the “Message” field, type in a description of what the change will do once applied. README.md Ctrl+S Shift+Ctrl+G

Writing “good” commit messages is an art in itself. If you want to get down that rabbit hole, head over to this page (if you make using Git a habit, reading that guideline is time well-spent).

Many practitioners adhere to three basic guidelines:

Keep the commit message short, not longer than 60 characters.

Write in present tense what the commit does to the repository.

It follows from 1.+2. that an individual commit contains focused, incremental work: it won’t turn the entire repository upside-down.

Press + / + to commit the change. You will be asked whether all changes shall be staged Ctrl+Enter and committed in one go. Agree to this suggestion. Now you have updated the local repository with the change to the file. README.md

Next, click the “Sync Changes” button and confirm: this will pull any changes from the remote repository to your local repository (there are none at this moment), and in a second step push your local commit to the remote repository (the change you just committed).

Return to the remote repository on GitHub (refresh the browser tab) to confirm that your commit has been pushed from local repository to remote repository.

3.2. Apply a quick fix to a committed change

At times, after a commit you notice a small mistake that you want to fix quickly - maybe a typo, or you want to include another file to the commit. Fortunately, Git lets you easily change the last commit:

Just fix whatever needs fixing. On the VS Code “Source Control” side pane click on the small “...” icon (“More Actions...”), then “Commit” and “Commit All (Amend)”. An editor tab opens that lets you change the commit message. Close the tab to commit the fix.

To perform the same action with Git terminal commands, first stage the change:

git add .

Next, commit with the flag:--amend

```
git commit --amend
```

An editor tab (or editor window) opens that lets you change the commit message. Close the editor tab (or window) to commit the fix.

3.3. Restore to an earlier repository state

Since Git keeps track of all the changes applied to the repository, it can automatically create a commit for you that restores the repository to an earlier state. (Think of this as a big undo button.)

Suppose we want to undo a change we applied before, say to the file. First, open that file.  README.md

From the VS Code "Explorer" side pane (++ / ++), open the "Timeline" section, then "Filter Timeline" for "✓ Git History".`ShiftCtrlEShift⌘E`

Click on an earlier commit to open a side-by-side comparison between current and older state (i.e., the diff view). If you found the correct commit, right-click on the timeline item and select "Copy Commit ID".

Open a terminal with ++ / ++, . Revert to the previous state with , for example:`ShiftCtrlPShift⌘P`terminal create new terminalgit revert {commit ID}

```
git revert c6e0eed92329d14417faf7c26b3ca8fd7a1c9ccb
```

Obviously, the commit ID will be different in your case.

An editor tab will open that lets you change the proposed commit message. In most cases you want to simply close the editor. Finally, a new commit is created with all changes applied such that in effect you have gone back in history.

The feature is very strong motivation to keep individual commits focused, often limited to just one file.`git revert`

If you want to restore just a particular file out of a commit that affects multiple files, call . With the flag, Git computes the changes and stages them, but does not commit them yet. This gives you the chance to un-stage and discard any changes that you don't want applied, before confirming the commit.`git revert -n {commit ID}-n`

4. Apply a basic GitHub workflow in VS Code

4.1. A basic GitHub workflow

GitHub adds a layer of collaboration functions on top of Git. Consider the following diagram:

```
main
feature
joe commit m-1
joe commit m-2
jane commit f-1
joe commit f-2
pull-request-1
```

rick review f-1
comment
jane review f-2
rick review f-3
ready to merge
v1

In its most basic form, the GitHub collaboration workflow discourages direct commits to the remote repository. Instead, the workflow suggests to

create a branch⁶ (e.g., called feature)
make one or multiple commits to that branch,
file a pull request⁷ to propose applying the branch changes to the branch, main
discuss the pull request, possibly making additional commits in the process,
(dis-)approve to merge⁸ the branch commits with the branch, and finally main
apply the change and delete the branch. feature

It is good practice to create and maintain a file that specifies the particular collaboration workflow for your repository. The Mozilla Science Lab has published a great instruction to get you started on this important topic.  CONTRIBUTING.md

4.2. Create a branch and create a commit

In VS Code, press ++ / ++ to open the command palette, then start typing . Select the command (not). In the following prompt, provide a name, e.g., . Accept the command by pressing .
ShiftCtrlPShift⌘P git create branch Git: Create Branch Git: Create Branch From ...feature Enter

You have just created a new branch and switched to it. By default, any commits will now be made to this branch. To try this out, change the file again and go through the same steps as before (you will notice that instead of “Sync Changes” the button will read “Publish Branch”).  README.md

4.3. File a pull request

For this step and the following ones, the “GitHub Pull Requests and Issues” extension must be installed and activated. Alternatively, open a browser tab pointed to the remote repository to carry out the same actions via the GitHub web application.

With the “Source Control” side pane still active, click on the small icon with the tool-tip “Create Pull Request”. Alternatively, open the command palette and start typing , then select .
create pull request GitHub Pull Requests: Create Pull Request

The “GitHub Pull Request” side pane opens. Verify the correct source and destination branches are selected, complete pull request title and description, and finally create the pull request.

4.4. Review and accept the pull request, delete the branch

Since you just created the pull request, the newly created discussion board opens automatically. To access pull requests filed by others (or review your own pull requests later on), open the “GitHub Pull Requests and Issues” extension, and navigate to the pull request of interest.

The discussion board gives you multiple options, among others:

assign roles such as reviewers to the pull request,
leave a comment,
review the proposed changes in a diff view,
close the pull request (thus rejecting it), and

merge the pull request (thus accepting it).

For now, simply click on “Merge Pull Request”, review and possibly adapt title and description to this action.

Next, click “Create Merge Commit” to execute the branch merge.

Finally, click on “Delete branch...” and confirm the selection to delete the branch you created before.

Congratulations, you have walked through a basic GitHub workflow, all from the comfort of Visual Studio Code.

Suppose you manage pull request via the GitHub web application, not VS Code plugin as described above.

When you delete the branch, it gets deleted only remotely (i.e. on GitHub), not locally on your copy of the repository. To delete the local branch:

On the Source Control pane, make sure you are not on the branch to be deleted (e.g., you are on not mainfeature)

Press ++ / ++ to open the command palette, start typing ShiftCtrlPShift⌘Pggit delete branch

From the drop-down, choose the branch to be deleted, e.g., , and confirm your choice.feature

Mehr anzeigenWeniger anzeigen

Sehr gute Frage 🍌 und hier ist die **ehrliche Antwort**:

👉 **Du kannst den ganzen Abschnitt 3 + 4 größtenteils skippen (für jetzt)**

Warum?

Du hast schon:

-  Commit gemacht
-  Push gemacht
-  Repo funktioniert
-  Git verstanden (Basics)

👉 **3.1 hast du also schon gemacht → automatisch ✓**

Was du NICHT brauchst (jetzt)

3.2 Amend Commit

→ nur für kleine Korrekturen

👉 später lernen, jetzt egal

❌ 3.3 Revert (zurückgehen)

→ fortgeschritten

👉 brauchst du erst bei Fehlern

❌ 4.x Branches + Pull Requests

→ wichtig für Teams

👉 aber:

➡ Du arbeitest alleine → aktuell unnötig

✅ Was du WIRKLICH brauchst

👉 Nur diesen Workflow:

Bash

```
git add .  
git commit -m "message"  
git push
```

👉 Das reicht für 90% am Anfang

🔥 Was du stattdessen JETZT machen solltest

👉 1. (falls noch nicht gemacht)

Bash

```
pip freeze > requirements.txt  
git add requirements.txt  
git commit -m "add requirements"  
git push
```

👉 2. Jetzt kommt der wichtige Teil 🚀

👉 Flask richtig aufbauen (Website)

Das ist viel wichtiger als Git-Theorie jetzt



Zusammenfassung

Abschnitt	Machen?
3.1 Commit	✗ schon gemacht
3.2 Amend	✗ skip
3.3 Revert	✗ skip
4.x Branches	✗ später

Fazit

- 👉 Du bist schon weiter als die Anleitung 😊
- 👉 Jetzt bringt dir Git nicht mehr viel → **jetzt kommt Coding**

Nächster sinnvoller Schritt:

👉 Flask Website mit HTML bauen

Sag einfach:

👉 „Website bauen“

Dann zeig ich dir Schritt für Schritt:

- templates
- HTML
- mehrere Seiten
- sieht richtig professionell aus 😎

GitHub Pages

GitHub Pages is a webhosting service for static web pages - this is useful to host documentation, blog posts, etc. By default, web pages are available under the domain `hwrberlin.github.io/fswd/{account}.github.io/{repository}/`

The basic workflow to publish a web page via GitHub Pages is:

Enable the “Pages” feature on an existing GitHub-hosted repository

Configure the web page, e.g., select a theme

Upload your content to the repository via the usual Git-based mechanisms

After every change to the repository, GitHub spins up Jekyll to convert the content into an HTML page

After this task has completed, the new (or changed) web page is publicly available

In the following, we will go through these steps on the example of the example repository for this course.

The basic file structure for your own GitHub Pages setup is available on the fswd-app repository.

First, git clone the repository.

Next, run to obtain the source code at tagged location `.git checkout docsdocs`

Alternatively, download a .zip file containing the source code from the GitHub web app.

You may want to copy the folder from into your own project repository. `docs/fswd-app`

5.1. Enable Pages and configure the web page

From the GitHub web interface, enter the repository's  Settings. Under Pages, choose these settings:

Source: Deploy from a branch

Branch: , then choose the folder `main` `docs/`

Click and wait a minute or so for the initialization to complete. Save

Enable GitHub Pages

With this step completed, GitHub will look for any changes in your repository's folder. After every change, a workflow kicks in to automatically update your web page. `docs/`

If you use code from the example repository as starting point for your own repo, the folder will already contain a file structure. This includes the file , which tells Jekyll how you want your web page generated: `docs/ _config.yml`

title: [Project Name]

remote_theme: just-the-docs/just-the-docs

mermaid:

version: "11.12.0" # Pick version from <https://cdn.jsdelivr.net/npm/mermaid/>

callouts:

tip:

title:  Tip

color: purple

info:

title: i Info

color: blue

attention:

title:  Attention

color: yellow

download:

title:  Download

color: grey-dk

Like many modern configuration files, uses the popular YAML syntax. YAML is similar to JSON, but more expressive. Also, YAML uses the "indentation instead of curly brackets" style you already know from Python. `_config.yml`

For example, with we tell the site generator to use the just-the-docs/just-the-docs repository as our template source. `remote_theme`

The Jekyll site generator uses Liquid as template engine: Liquid is like Jinja, but written in Ruby, not Python.

We also defined several callout types, which is a feature specific to the chosen template.

To start with, it is sufficient to just update the value and keep the rest as-is.

title: [Project Name] is a placeholder, i.e. replace like so: .title: Todo-App

If you want to adjust additional settings and need help later, you will usually look on these places for documentation (in this order):

The configuration reference of the chosen template

The configuration reference of the Jekyll application

Mehr anzeigen Weniger anzeigen

Sehr gute Frage 🙌 und hier ist die **klare, ehrliche Antwort**:

👉 **Das ist NICHT dein nächster Schritt** (für dein aktuelles Projekt)

🧠 Warum nicht?

! GitHub Pages = nur für statische Webseiten

Das bedeutet:

- ☒ HTML
- ☒ CSS
- ☒ einfache Seiten

Aber:

- ☒ **KEIN Flask**
- ☒ kein Python Backend
- ☒ kein `app.py`

🚫 Dein Projekt ist:

👉 eine **Flask App (Backend + Server)**

➡ Die läuft nur lokal oder auf Servern wie:

- Render
- Railway
- Heroku

👉 **NICHT** auf GitHub Pages

Fazit:

Tool	Für dich sinnvoll?
GitHub Pages	✗ NEIN
Flask App lokal	✓ JA
Flask online deployen	✓ später

Wann GitHub Pages sinnvoll ist

👉 Wenn du z. B.:

- Portfolio Seite baust
- Dokumentation schreibst
- nur HTML hast

Was du stattdessen machen solltest

Jetzt kommt der **richtige nächste Schritt für dich**:

Flask Website richtig bauen

- mehrere Seiten
- HTML Templates
- echte Website

Später (wenn du willst)

Dann kannst du:

👉 deine Flask App online stellen
(z. B. mit Render)

Kurz gesagt:

👉 GitHub Pages → skippen

👉 Flask weiterbauen → richtig

Sag einfach:

👉 „Flask Website bauen“

Dann zeig ich dir Schritt für Schritt:

- Templates
- HTML Seiten
- Navigation

👉 ab da wird dein Projekt richtig cool 😄